PHP was invented to handle HTTP and HTML and offers a lot of built-in functionality for running in a web server environment. Some of what we'll be looking at will require that the code is running within the context of an HTTP request, so let's have a quick look at how to get a web server up and running with PHP.

There are multiple web servers available, the two with the greatest market-share for PHP today are Apache and Nginx. We'll go with Nginx because it is relatively straightforward to install and configure compared to Apache. To install Nginx you have similar options to installing PHP (package manager, Docker, etc.) and for the sake of brevity – not to mention cross-platform compatibility – we shall use Docker containers. We'll also learn a little bit more about how to orchestrate containers so they can work together, so it's real value for money!

Nginx, unlike Apache, does not permit PHP to run as part of its process (via your system's linker). Instead it uses a standard for web-server interprocess communication called FastCGI, which is a binary protocol that permits long-running processes to service a series of requests. We will configure Nginx so that when it receives an HTTP request for a resource ending in `.php` the request is forwarded to a FastCGI compatible instance of PHP. We've already seen in the previous section how to get the official PHP CLI image running from the default Dockerhub repository, but this time we're going to need PHP's FPM (FastCGI Process Manager) image. Naturally there's an official image for Nginx too. Remember that Docker is about containing a process, not an entire system, so we'll need two containers: one for `nginx` and one for `php-fpm`.

Nginx (via Docker Orchestration)

Docker has a handy command for configuring multi-container projects: `docker compose`. Some systems also have a `docker-compose` (aka "compose v1") command available, but `docker compose` (aka "compose v2") is the shiny new way of doing things and should work everywhere. If you're working with an older version of Docker and `docker compose` isn't available, use the old `docker-compose` binary (you might need to install a separate package to get it). For our purposes the two are interchangeable.

We'll configure our two containers with a special config file – `docker-compose.yaml` – and we'll also need another special file – `Dockerfile` – for making changes to the base images before the containers are started. To enable `nginx` in one container to work correctly with `php-fpm` in another we'll need the following criteria met: